

Web Storage

Powerful Client-Side Data Persistence

Web Storage or Client-Side Storage

- Client-side storage refers to the ability of web applications to **store data directly in a user's browser**.
- It allows for **faster access** to data and enhances user experience by enabling features like offline access, user preferences, and session management.
- It is a new HTML5 API offering important benefits over traditional cookies.
- A **built-in browser tool** to save small data on the user's device (like a notepad).
- Stores data as **key-value pairs** (e.g., "theme" : "dark").
- Only text can be stored (no images/files).
- The data being stored can be accessed using JavaScript, which gives you the ability to leverage client-side scripting to do many things
- There are two Web Storage objects:
 - 1) **localStorage**
 - 2) **sessionStorage**

Welcome

</> Elements

Console

Sources

Network

Performance

Memory

Application

+

Application

Manifest

Service workers

Storage

Storage

▶ Local storage

▶ Session storage

Extension storage

▶ IndexedDB

▶ Cookies

Private state tokens

Interest groups

▶ Shared storage

▶ Cache storage

Storage buckets

Local storage

On this page you can view, add, edit, and delete local storage key-value pairs. [Learn more](#)

HTML5 Web Storage Objects

- HTML web storage provides two objects for storing data on the client:
 - **window.localStorage** - stores data with no expiration date
 - **window.sessionStorage** - stores data for one session (data is lost when the browser tab is closed)

Local Storage

- Local storage allows developers to store and retrieve data in the browser.
- The data stored in local storage will not expire. This means the data will persist even if the tab or the browser window is closed.
- Data stored here can be accessed throughout a particular domain.
- DOM Window localStorage is provided by Browser and it allows us to store data as key-value pairs in our web browser using an object.
- The localStorage is the read-only property of the window interface

Web Storage objects

- `localStorage` - to display the `localStorage` object
- `localStorage.clear()` - to remove everything in the local storage
- `localStorage.setItem()` - to store data in the `localStorage`. It takes a key and a value parameters.
- `localStorage.getItem()` - to display data stored in the `localStorage`. It takes a key as a parameter.
- `localStorage.removeItem()` - to remove stored item form a `localStorage`. It takes key as a parameter.
- `localStorage.key()` - to display a data stored in a `localStorage`. It takes index as a parameter.

- It is important to note that local storage only stores strings so, if you wish to store objects, lists, or arrays, you must convert them into a string using **JSON.stringify()**.

JSON.stringify & JSON.parse

- The JSON object, available in all modern browsers, has two useful methods to deal with JSON-formatted content: parse and stringify.

JSON.stringify()

- JSON.stringify() takes a JavaScript object and transforms it into a JSON string.

Eg:

```
let userObj = {  
    name: "Sammy",  
    email:  
    "sammy@example.com",  
    plan: "Pro"  
};
```

```
let userStr = JSON.stringify(userObj);
```

```
console.log(userStr);
```

Executing this code will produce the following output:

Output

```
{"name":"Sammy","email":"sammy@example.com","plan":"Pro"}
```

JSON.parse()

- JSON.parse() takes a JSON string and transforms it into a JavaScript object.

Eg : `let userStr =
'{"name":"Sammy","email":"sammy@example.com","plan":"Pro"}';`

`let userObj = JSON.parse(userStr);
console.log(userObj);`

Executing this code will produce the following output:

Output

```
{  
  name: 'Sammy',  
  email: 'sammy@example.com',  
  plan: 'Pro'  
}
```

- You should only use local storage when storing insensitive information.
- This is because third-party individuals can easily access the information.
- Local storage can help in storing temporary data before it is pushed to the server.
- It is important to clear the local storage once this operation is completed.

The major limitations of local storage are:

- Insecure data.
- Synchronous operations.
- Limited storage capacity.

key()

- This method is used to retrieve a value/string from a specific location.
- The index can be passed into the key() function as a parameter.
- The key() can also be used in a loop statement to retrieve all the items in the local storage.

Eg: `var answer = localStorage.key(1);`

`// this statement will retrieve the value of the second item in localStorage.`

setItem()

- This function is used to store items in local storage.
- As mentioned before, we must stringify objects before we store them in the local storage.

Eg: `window.localStorage.setItem("grade","One");`

//in this case, the `grade` is the key while `One` is the value.

```
Eg: const Car = {  
    brand:"Suzuki",  
    color:"white",  
    price:10000  
}
```

```
localStorage.setItem('car', JSON.stringify(Car));
```

getItem()

- This function is used to access or retrieve the data in the local storage.
- The method takes in a key as a parameter.
- It then extracts the required value from the localStorage.
- For example,

```
localStorage.getItem('grade');
```

or

```
JSON.parse(localStorage.getItem('car'));
```

removeItem()

- This method is used to delete an item from local storage.
- The removeItem() method requires a key as a parameter.

```
localStorage.removeItem('brand');
```

clear()

- This method is used to clear all values stored in local storage.
- It does not require any parameters.

`localStorage.clear()`

Session Storage

- Session storage allows developers to **store and retrieve data in the browser** for a single session.
- The data stored in session storage is **temporary** and will be **cleared automatically when the tab or browser window is closed**.
- Data stored here is **accessible only within the same tab and same origin** (same domain, protocol, and port).
- DOM Window sessionStorage is provided by the **browser** and is used to store data as **key-value pairs**.
- The sessionStorage is a **read-only property** of the window interface.
- It is ideal for storing **temporary data** like:
 - Step-by-step form data
 - Filter settings in a single session
 - Temporary messages or state

Cookies

- Cookies allow developers to **store small amounts of data (around 4KB)** in the browser.
- Unlike localStorage or sessionStorage, cookies are **automatically sent to the server** with every HTTP request.
- Cookies can be used for:
 - **Session management** (like login state)
 - **User preferences**
 - **Tracking and analytics**
- Cookies can have an **expiration time** (using expires or max-age).
If not set, the cookie expires when the browser is closed.
- Stored as a **single string of key-value pairs** in document.cookie.
- Limited in size (≈ 4096 bytes) and number (≈ 20 – 50 cookies per domain).

Comparison

Feature	LocalStorage	SessionStorage	Cookies
Size Limit	~5MB	~5MB	~4KB
Expiration	Manual	On tab close	Manual / session
Sent to server	No	No	Yes
Accessible by JS	Yes	Yes	/ (HttpOnly)
Domain Specific	Yes	Yes	Yes
Use Case	Persistent prefs	Temporary session	Authentication

Task

- 1) What is local storage, and how is it different from session storage?
- 2) What is the size limit for local storage and session storage in most browsers?
- 3) Can you describe how data is stored in local storage? Is it secure?
- 4) Explain a scenario where you would prefer session storage over local storage.
- 5) How would you store an array or object in local storage?
- 6) How would you handle data expiration in local storage since it doesn't expire by default?
- 7) What are the differences between cookies, local storage, and session storage? When would you use each?

TASK

Task: Working with Cookies in JavaScript

Write JavaScript code to perform the following cookie operations:

- Create a cookie named userToken with the value "abc123" that expires after 7 days.
- Read and display all cookies in the console.
- Delete the userToken cookie.

TASK

- **Explore Other Browser Storage Options**

Research and briefly demonstrate these of the following advanced browser storage mechanisms:

- **IndexedDB**
- **Cache Storage (Service Workers)**
- **FileSystem Access API**

Thank You